

```
>>> seaDuck.floatOnWater()
Floating...
```

Multiple inheritance is straightforward as long as the parent classes' method names are distinct and don't overlap. These sorts of classes are called *mixins*. (This is just a general term for a kind of class; Python has no `mixin` keyword.) But what happens when we inherit from multiple complicated classes that do share method names?

For example, consider the `MiniBoard` and `HintTTTBoard` tic-tac-toe board classes from earlier in this chapter. What if we want a class that displays a miniature tic-tac-toe board and also provides hints? With multiple inheritance, we can reuse these existing classes. Add the following to the end of your `tictactoe_oop.py` file but before the `if` statement that calls the `main()` function:

```
class HybridBoard(HintBoard, MiniBoard):
    pass
```

This class has nothing in it. It reuses code by inheriting from `HintBoard` and `MiniBoard`. Next, change the code in the `main()` function so it creates a `HybridBoard` object:

```
gameBoard = HybridBoard() # Create a TTT board object.
```

Both parent classes, `MiniBoard` and `HintBoard`, have a method named `getBoardStr()`, so which one does `HybridBoard` inherit? When you run this program, the output displays a miniature tic-tac-toe board but also provides hints:

```
--snip--
      X.. 123
      .O. 456
      X.. 789
X can win in one more move.
```

Python seems to have magically merged the `MiniBoard` class's `getBoardStr()` method and the `HintBoard` class's `getBoardStr()` method to do both! But this is because I've written them to work with each other. In fact, if you switch the